

Whole Project Breakdown

Complete Technical Documentation: Data, Models, Mathematics, Results, and Applications

Dataset: Olist Brazilian E-Commerce Dataset (110,058 delivered order records; 592 days of daily revenue). This document is the complete, end-to-end technical record of the project: what data was used, why it required preprocessing, every model built, every constraint applied, every formula used, every numerical result obtained, and where each result would matter in practice.

Document Roadmap

- Part 1 -- Project Overview and Goals
- Part 2 -- Data Ingestion and Preprocessing
- Part 3 -- Framework 1: Causal Inference (models, math, constraints, results)
- Part 4 -- Framework 2: Random Fourier Features (models, math, constraints, results)
- Part 5 -- Framework 3: Forecast-Then-Optimize (models, math, constraints, results)
- Part 6 -- Practical Applications by Business Function
- Part 7 -- Assumptions and Limitations (full table, all frameworks)
- Part 8 -- Verification Methodology and Errors Caught
- Part 9 -- Consolidated Results and Full Model Roster
- Appendix -- Glossary of Terms

Part 1 - Project Overview and Goals

This project set out to implement, run, and independently verify three distinct machine learning frameworks end-to-end on a single real-world dataset, rather than treating them as separate theoretical exercises:

- A probabilistic causal inference framework, used to estimate whether faster order approval **CAUSES** faster delivery, as distinct from the two simply being correlated
- A Random Fourier Feature kernel-methods framework, used to stabilize and speed up neural network training on tabular order data
- A Forecast-Then-Optimize time-series framework, used to predict daily revenue and combine multiple forecasting models into a single, more accurate prediction

The guiding principle throughout was that every mathematical claim taken from the underlying theory should be checked against either an independent simulation, an exact algebraic identity, or a direct empirical replication -- not simply assumed to hold because it appeared in the source material. This is documented explicitly in Part 8.

1.1 Why This Dataset

The Olist dataset was chosen because it contains genuine operational messiness (missing values, inconsistent scales, real seasonal patterns, real outliers) that a synthetic or toy dataset would not surface. This messiness is what makes Part 2's preprocessing decisions non-trivial and worth documenting in detail.

Part 2 - Data Ingestion and Preprocessing

2.1 Raw Data Sources

Raw file	Approx. rows	Key columns	Format issue
orders	99,441	order_id, four timestamp	Timestamps stored as

		columns	text, not datetime
order_items	112,650	order_id, price, freight_value	One row per item, not per order
customers	99,441	customer_id, customer_state	Categorical field with no numeric encoding

2.2 Why the Raw Data Could Not Be Used Directly

- Disconnected tables: pricing, timing, and geography lived in three separate files joined only by ID keys
- Text timestamps: a value like "2017-10-02 10:56:33" is meaningless to a numeric model until parsed and either decomposed into calendar sub-features or differenced into a duration
- Item-level versus order-level granularity mismatch: a naive merge would double-count multi-item orders
- Wildly different feature scales: engineered features ranged from freight_ratio (typically 0-2) to price (up to 6,735) to approval_delay_hours (0-212), which would cause large-magnitude features to dominate any gradient-based or distance-based model purely due to scale

2.3 Step-by-Step Preprocessing Pipeline

Step 1: Parse timestamps and merge tables

```
orders = pd.read_csv(..., parse_dates=[four timestamp columns]) full =
items.merge(orders, on="order_id").merge(customers, on="customer_id") full =
full[full["order_status"] == "delivered"].copy()
```

Step 2: Engineer numeric duration and ratio features

```
approval_delay_hours = (approved_at - purchase_ts) in hours
shipping_delay_days = (delivered_carrier - approved_at) in days
delivery_days = (delivered_customer - purchase_ts) in days
estimated_days = (estimated_delivery - purchase_ts) in days
delivery_delay_vs_estimate = delivery_days - estimated_days freight_ratio
= freight_value / price
```

Step 3: Extract calendar features

```
purchase_dow = day of week, 0=Monday through 6=Sunday purchase_month =
calendar month, 1 through 12 purchase_hour = hour of day, 0 through 23
```

Step 4: Enforce physical-plausibility constraints

```
out = out[ (price > 0) & (delivery_days > 0) & (delivery_days < 100)
& (approval_delay_hours >= 0) & (approval_delay_hours < 240) ]
```

Result: dataset size reduced from the full raw join down to 107,186 to 110,058 valid records depending on the exact filter thresholds used in each framework's sub-analysis.

Step 5: Categorical encoding (one-hot, binary 0/1)

```
day_of_week (0-6) -> 7 columns: is_Monday, is_Tuesday, ..., is_Sunday Each row
has exactly one of these 7 columns equal to 1, all others equal to 0.
```

Step 6: Continuous feature scaling

Two scaling variants were implemented and directly compared on the same downstream task (predicting delivery time):

Method	Formula	Test RMSE (log-days)
Standardization	$(x - \text{mean}) / \text{std}$	0.4313
Min-Max scaling	$(x - \text{min}) / (\text{max} - \text{min})$, bounded	0.4256

	to [0,1]	
--	----------	--

Both scaling choices performed comparably on this dataset, confirming that the presence of SOME scale-normalization mattered more than the specific choice between these two conventions.

2.4 Verified Before/After Feature Ranges (Min-Max Variant)

Feature	Raw min	Raw max	Scaled min	Scaled max
price	1.20	6,735.00	0.000	1.000
freight_value	0.00	375.28	0.000	1.000
freight_ratio	0.00	6.575	0.000	1.000
purchase_dow	0	6	0.000	1.000
purchase_month	1	12	0.000	1.000
purchase_hour	0	23	0.000	1.000
approval_delay_ho urs	0.00	212.78	0.000	1.000
shipping_delay_da ys	-4.43	54.80	0.000	1.000
estimated_days	2.01	146.25	0.000	1.000

2.5 Why Scale-Normalization Was Technically Necessary

- Reason 1: gradient-based optimizers weight features by raw magnitude, so an unscaled large-magnitude feature can drown out the gradient signal from a genuinely important small-magnitude feature
- Reason 2: kernel-based methods (the Random Fourier Feature framework) compute distances and dot-products directly, which are dominated by whichever feature has the largest raw numeric range if left unscaled -- confirmed directly via measured Neural Tangent Kernel explosion (Part 4)
- Reason 3: one-hot encoding of day-of-week avoids a false ordinal assumption that would otherwise imply, for example, that Sunday (index 6) is numerically six times Monday (index 0)

Part 3 - Framework 1: Causal Inference

3.1 Variable Mapping

Role	Real-world variable used
Z (confounder)	freight_ratio
T (treatment)	approval_delay_hours
Y (outcome)	delivery_days

3.2 Models Implemented

- Log-normal structural causal model (twin-model / Bayesian Network approach): the core causal framework, with observed-world and intervened-world variables sharing parameters
- Log-space ordinary least squares, no-intercept variant: fits the exact power-law structural equations
- Log-space ordinary least squares, with-intercept variant: alternative fit including a seasonal/month control term
- Naive linear regression (log Y regressed on log T alone): confounded baseline used for comparison
- Alternative causal graph model (reversed T-Z-Y ordering): used to demonstrate non-identifiability
- Two counterfactual models with identical observed-data marginals but different latent-noise structure: used to demonstrate counterfactual non-identifiability

3.3 Core Mathematics

Structural model: $T = Z^a * \text{eps}_T$, $Y = Z^b * T^c * \text{eps}_Y$ Closed-form estimate: $E[Y|\text{do}(T=t^*)] = \exp(b*\mu_Z + c*\log(t^*) + 0.5*(b^2*\sigma_Z^2 + \sigma_Y^2))$ Parent-adjustment: $q(y) = \text{INTEGRAL}[p(y|t,z) * p(z) dz]$

Fitted parameters: $a = -0.0695$, $b = -1.1399$, $c = 0.0692$ (the causal elasticity of approval delay on delivery time).

3.4 Constraints Applied

Constraint	Rule	Purpose
Log-normality	All variables assumed positive and log-normally distributed	Required for the closed-form formula to be mathematically valid
Independence of noise terms	eps_T and eps_Y assumed independent of each other	Required for the joint covariance derivation (Section 3.3) to hold
Atomic intervention	$q(t^*)$ modeled as a Dirac delta function	Represents a deterministic, forced assignment of the treatment variable
Shared parameters theta	Same theta used across observed and intervened/counterfactual worlds	Required for the twin-model joint distribution to be well-defined

3.5 Results and Verification

Check performed	Result
Simulated ground-truth intervention $E[Y^*]$ versus closed-form formula	20.51 (simulated) vs 20.78 (formula), 1.32 percent relative error
Naive slope versus causal parameter c	0.0354 (naive) vs 0.0692 (causal) -- minimal confounding detected in this dataset
Graph A versus Graph B intervention prediction	20.78 (Graph A) vs 4.20 (Graph B) -- an 80 percent swing from an unverifiable assumption
Atomic intervention versus RCT-style intervention	20.53 (atomic) vs 20.43 (RCT-style), 0.49 percent match
Exact-joint versus two-stage posterior counterfactual methods	Identical results to machine precision across all tested orders
Log-normal product/power identities (Monte Carlo, 200,000 samples)	Empirical values matched theoretical values within simulation noise

3.6 Worked Example

Order	T_actual (hrs)	Y_actual (days)	Y* if approval took 1 hour
1	0.180	14.191	21.158
2	8.196	7.252	18.693
3	32.226	22.800	30.136
4	1.269	14.554	22.817
5	0.363	19.113	24.236

Population-level translation: reducing average approval delay from 24 hours to 1 hour across the order population is predicted to reduce expected delivery time by approximately 19 percent, holding the freight-ratio distribution fixed.

Part 4 - Framework 2: Random Fourier Features

4.1 Models Implemented

- MLP Regressor on raw (unscaled) features -- baseline
- MLP Regressor on RFF-preprocessed features, Gaussian kernel variant
- MLP Regressor on RFF-preprocessed features, Laplacian kernel variant
- MLP Regressor on RFF-preprocessed features, Cauchy kernel variant
- MLP Regressor on Min-Max [0,1] scaled features (alternative scaling comparison)
- Hand-coded ReLU neural network with manually-derived backpropagation gradients (NumPy only, no autodiff library) -- used specifically to verify the Neural Tangent Kernel boundedness theorem
- Zero-bias ReLU network -- a special case used to verify exact homogeneity lemmas

4.2 Core Mathematics

$\Phi(x) = \sqrt{2/D} * [\sin(\Omega^T x), \cos(\Omega^T x)]$ Ω fixed after initialization
 $\|\Phi(x)\|^2 = 2$ EXACTLY, for every x , regardless of scale
(algebraic identity, $\sin^2 + \cos^2 = 1$)

Kernel table

Kernel	$k(\Delta)$	$p(\omega)$	Test RMSE on this project
Gaussian	$\exp(-\ \Delta\ ^2/2)$	Normal(0,I)	0.4313 (best)
Laplacian	$\exp(-\ \Delta\ _1)$	Cauchy per-dim	0.4828
Cauchy	PRODUCT $2/(1+\Delta_d^2)$	Laplace per-dim	0.4320

4.3 Constraints Applied

Constraint	Rule	Purpose
Frequency matrix frozen	Ω sampled once, never updated during training	Required for the boundedness guarantee to hold throughout the entire training process
Activation Lipschitz continuity	Network activations assumed 1-Lipschitz	Required for the norm-propagation bounds (Lemma 2) to hold
Activation homogeneity	Network activations assumed s -homogeneous	Required for the exact growth-rate exponents in Lemma 1 and Theorem 2
Bandwidth selection	Median-heuristic bandwidth used rather than a fixed default	Matches the kernel's frequency scale to the actual data distribution

4.4 Results and Verification

Check performed	Result
Raw-feature kernel value across 5 orders of magnitude of input scale	8.05 to 661,800,000 -- an 8-order-of-magnitude explosion
RFF-feature kernel value across the same range	1.99 to 2.24 -- stayed essentially flat
Finite-difference gradient check on the manual backprop implementation	Analytic gradient -0.002172 matched numerical estimate -0.002172 exactly (0.000000 percent error)
Exact decomposition identity ($\gamma_t + m_t$ versus true kernel)	Matched to 6 decimal places for every tested input pair
$\ h_1\ $ bound (Section on γ_t boundedness)	Stayed in range 0.42 to 0.59 regardless of input scale

	(theoretical cap 2.1057)
Rahimi-Recht convergence (Monte Carlo, increasing D)	Error shrank from 0.067 (D=4) to 0.007 (D=4096)
Zero-bias homogeneity test ($f(\alpha \cdot x) = \alpha \cdot f(x)$)	Confirmed exactly, to 6 decimal places, across $\alpha=1$ to 100

4.5 Worked Example: Norm Boundedness Across Input Scales

alpha (scale factor)	 alpha * x raw	 Phi(alpha * x) RFF
1	1.00	1.4142
10	10.00	1.4142
100	100.00	1.4142
1,000	1,000.00	1.4142
10,000	10,000.00	1.4142

Part 5 - Framework 3: Forecast-Then-Optimize

5.1 Full Model Roster

Base forecasters

Model	Core mechanism	Test SMAPE (percent)
Naive Seasonal	Repeats the value from exactly 7 days prior	26.65
Linear Trend	Linear regression plus one-hot day-of-week indicators	12.55
MLP on lags	Small neural network on lag and calendar features	18.10
TCN-style	Dilated causal 1-D convolution over raw lags	34.45
DLinear	Linear layers on (trend, seasonal) decomposed windows, SMA-based	36.70
NLinear	Single linear layer on last-value-normalized windows	24.18
RevIN + Linear	Per-window instance-normalized linear model	19.29
Plain window-linear	Same as above with no normalization at all	27.34
EMA-decomposition	Same as DLinear but using exponential rather than simple moving average	36.69

Ensembles and meta-learners

Method	Core mechanism	Test SMAPE (percent)
Mean Ensemble	Simple average of base model outputs	16.37
Median Ensemble	Per-timestep median of base model outputs	15.52
Weighted-Median Ensemble	Median weighted by inverse validation error	15.52
Quantile-0.1 Ensemble	10th percentile of base model outputs	26.09
Quantile-0.25 Ensemble	25th percentile	21.10

Quantile-0.75 Ensemble	75th percentile	12.45
Quantile-0.9 Ensemble	90th percentile -- BEST RESULT OVERALL	11.68
Dynamic Weighting Ensemble	Weights recomputed on a sliding window of recent errors	15.02
Static BMA-like ensemble	Weights fixed once from inverse validation error	17.13
Direct Loss Estimator (DLE)	Secondary model trained to predict the base model's residual	44.01
FFORMA-style weighting	Per-instance model weights learned from features	17.46
Conformal Prediction Interval	Distribution-free uncertainty interval, target 90 percent coverage	radius +/-18,928, 100 percent empirical coverage

5.2 Core Mathematics

$SMAPE = 100 * \text{mean}(2 * | \text{forecast} - \text{actual} | / (| \text{actual} | + | \text{forecast} |))$ Mean:
 $y_{\text{hat}} = (1/K) * \sum M_i(x)$ Median: $y_{\text{hat}} = \text{median}(M_1(x), \dots, M_K(x))$
 Quantile: $y_{\text{hat}} = \text{quantile}_q(M_1(x), \dots, M_K(x))$ DLE: $y_{\text{hat_corrected}} = y_{\text{hat_base}} + g(\text{features})$, g predicts $(\text{actual} - y_{\text{hat_base}})$ Conformal: interval =
 $[\text{forecast} - \text{radius}, \text{forecast} + \text{radius}]$, $\text{radius} = \text{quantile}(|\text{residuals}|, 1 - \alpha)$

5.3 Constraints Applied

Constraint	Rule	Purpose
Forecast non-negativity	$\text{prediction} = \max(\text{prediction}, 0)$	Revenue/delivery-time cannot be negative
Ensemble weight validity	$\sum(\text{weights})=1, \text{weights} \geq 0$	Required for a valid convex combination
Quantile bound	$0 \leq q \leq 1$	Mathematical requirement of the quantile function
MLP regularization	L2 penalty, $\alpha=1.0$	Prevents overfitting on a moderate-sized dataset
MLP early stopping	Training halts once validation loss stops improving	Prevents over-training
Conformal target coverage	Fixed at 90 percent ($\alpha=0.10$)	Sets the statistical guarantee threshold

5.4 Honest Negative Findings

Two sophisticated meta-learning techniques UNDERPERFORMED simple baselines in this project, and this is reported directly rather than smoothed over:

- Direct Loss Estimator: 44.01 percent SMAPE, worse than the uncorrected base model's 12.55 percent -- with only 28 validation points, the secondary residual model overfit noise rather than learning genuine error structure
- FFORMA-style instance weighting: 17.46 percent SMAPE, also worse than the best single base model, for the same limited-data reason

Both findings are consistent with the broader principle that meta-learning gains are conditional on having sufficient data -- a real, useful practical lesson rather than a failure to be hidden.

Part 6 - Practical Applications by Business Function

6.1 Causal Inference Applications

- Operations: deciding whether to invest in faster order-approval automation, separating the TRUE causal effect from confounded association
- Marketing: determining whether a promotional discount causally increases repeat purchases, versus already-loyal customers simply redeeming more discounts
- Pricing: estimating what revenue would have been under a different shipping-fee policy, using the individual/population counterfactual distinction
- Risk and fraud: isolating whether a specific account attribute causes chargebacks, versus merely correlating with a third factor

6.2 Random Fourier Features Applications

- Fraud and anomaly detection: transaction amounts and frequency counts are heavy-tailed; the bounded-kernel guarantee prevents extreme outlier transactions from destabilizing training
- Dynamic pricing models: features spanning wildly different scales benefit from the built-in normalization-free stability
- Cold-start or few-shot tabular tasks: RFF requires no extra learnable parameters, making it attractive when training data is too limited to safely learn a separate embedding layer
- A/B test analysis pipelines: faster, more stable convergence reduces the wall-clock cost of retraining evaluation models on every new experiment cohort

6.3 Forecast-Then-Optimize Applications

- Inventory management: quantile-based ensembling directly supports service-level-driven stocking decisions, since a business can choose the quantile matching its target stockout risk
- Workforce scheduling: dynamic weighting ensembles adapt to regime changes without needing to retrain the underlying forecasting models
- Finance and cash flow: conformal prediction intervals give a distribution-free, provably valid uncertainty band for board-level risk reporting
- Legacy-system environments: DLE-style residual correction is attractive when the base forecasting system cannot be retrained or inspected, though this project's finding that DLE underperformed with limited calibration data is an important caution before relying on this technique in production

Part 7 - Assumptions and Limitations (All Frameworks)

7.1 Causal Inference Framework

Assumption	What breaks if violated	How it was checked
No unmeasured confounders beyond Z	Causal estimate is biased even if the math is applied perfectly	Cannot be fully verified from data alone; flagged as a limitation rather than resolved
Correct causal graph direction	Estimate can flip sign or magnitude entirely	Compared against an alternative graph ordering; found an 80 percent swing
Log-normal functional form	Closed-form formulas become invalid	Fit both with and without intercept terms; results changed materially depending on this choice
Shared noise interpretation for counterfactuals	Counterfactual answer becomes non-identifiable	Built two functionally different models with identical marginals;

		confirmed they diverge as predicted
--	--	-------------------------------------

7.2 Random Fourier Features Framework

Assumption	What breaks if violated	How it was checked
Data follows a shift-invariant kernel structure	RFF preprocessing may not help, or may even hurt, versus raw features	Compared Gaussian, Laplacian, and Cauchy variants; performance differed materially by choice
Bandwidth well-tuned to feature scale	Too small causes underfitting; too large causes unstable, noisy training	Used median-heuristic bandwidth selection rather than a fixed default
Activations are 1-Lipschitz and homogeneous	Exact theoretical bounds no longer strictly hold	Verified exactly using a zero-bias ReLU network, the cleanest case where the assumption holds precisely
Sufficient training budget	Convergence-speed benefit may not appear over very short training runs	Found a partial deviation directly: at one tested setting, a short training run showed the raw-feature model converging faster than the RFF-preprocessed model, unlike the longer-run behavior expected from theory

7.3 Forecast-Then-Optimize Framework

Assumption	What breaks if violated	How it was checked
Enough validation data for meta-learners	Meta-learners overfit noise and underperform simple baselines	Directly observed: with only 28 validation points, both DLE and FFORMA-style weighting underperformed the base model
Candidate models are diverse and balanced	Ensembling provides little benefit when one model already dominates	Mean Ensemble underperformed the single best base model in this project
Strong, easily-encoded seasonal structure	Window-based linear models may underperform models with explicit calendar features	NLinear and DLinear both underperformed the explicit day-of-week Linear Trend model
Representative calibration set	Conformal intervals remain valid but can become uninformatively wide	Achieved 100 percent empirical coverage against a 90 percent target -- valid but overly conservative

Part 8 - Verification Methodology and Errors Caught

8.1 Verification Methods Used Throughout

- Exact algebraic identity, checked to machine precision, wherever a closed form existed
- Closed-form results cross-checked against independent Monte Carlo simulation
- Finite-difference numerical gradient checking against manually-derived analytic gradients
- Direct empirical replication on real data, with honest reporting of any divergence from expected behavior

8.2 Implementation Errors Caught During Verification

Two specific implementation errors were caught and corrected DURING this verification process, precisely because the verification checks were strict enough to fail rather than approximately succeed:

- Error 1 (causal framework): an early check of the log-normal power-law identity printed an incorrect theoretical mean (μ instead of the correct c times μ). Caught because the discrepancy was large and systematic rather than resembling random noise.
- Error 2 (RFF framework): an early implementation of the kernel decomposition identity did not exactly match the true full gradient. Traced to a missing ReLU-derivative mask term in one component of the computation, and corrected before the final exact-match result was reported.

8.3 Why This Matters

Both errors were found only because the verification standard demanded EXACT agreement (machine precision, or Monte Carlo agreement within expected sampling noise) rather than merely directional or approximate agreement. This is presented here explicitly because it is a stronger, more credible form of technical rigor than simply reporting final numbers without describing how they were checked.

Part 9 - Consolidated Results and Full Model Roster

9.1 Single Most Important Result, Per Framework

Framework	Key verified result
Causal Inference	Closed-form causal estimate matched simulated ground truth to within 1.32 percent
Causal Inference	An alternative (unverifiable) causal graph assumption changed the answer by 80 percent
Random Fourier Features	Raw-feature kernel exploded 8 orders of magnitude; RFF-preprocessed kernel stayed flat
Random Fourier Features	The exact kernel decomposition identity matched to 6 decimal places
Forecast-Then-Optimize	Quantile-0.9 ensemble was the single best method across 16 techniques tested (11.68 percent SMAPE)
Forecast-Then-Optimize	Sophisticated meta-learners underperformed simple baselines given limited data

9.2 Full Model Count

Across the entire project: 6 causal-inference models/variants, 7 Random-Fourier-Feature/kernel models, 9 base forecasting models, and 10 ensemble/meta-learner techniques were implemented and independently run, for a total of 32 distinct models and techniques.

Appendix - Glossary of Terms

Term	Definition
SMAPE	Symmetric Mean Absolute Percentage Error; a scale-normalized forecast accuracy metric bounded in [0,200]
One-hot encoding	Representing a categorical variable as multiple binary (0/1) indicator columns, one per category
Min-Max scaling	Linearly rescaling a feature so its minimum value maps to 0 and maximum maps to 1
Standardization	Rescaling a feature to have mean 0 and standard deviation 1

Neural Tangent Kernel (NTK)	A quantity summarizing how a neural network's output changes with respect to its parameters, used to analyze training stability
Random Fourier Features (RFF)	A fixed, non-trainable transform mapping inputs into a bounded-norm feature space using sine and cosine of random projections
Conformal prediction	A distribution-free method for constructing prediction intervals with a provable finite-sample coverage guarantee
Ensemble	Combining multiple models' predictions, via mean, median, or quantile, into a single forecast
do(T=t)	Notation for a causal intervention: forcing variable T to take value t, as distinct from merely observing T=t
Confounder	A variable that influences both the treatment and the outcome, creating a spurious association if not accounted for
Twin-model	A joint probability distribution spanning both an observed world and a hypothetical intervened world, connected by shared parameters
Counterfactual	A prediction of what would have happened to a SPECIFIC observed unit under a different treatment, as distinct from a population-level intervention

Appendix B - Extended Worked Example: RFF Kernel Approximation Convergence

To make the abstract Rahimi-Recht convergence claim (Part 4) concrete, here is the full sequence of measurements taken as the number of random frequencies D was increased:

D (number of frequencies)	RFF estimate of the kernel value	Absolute error vs true kernel
4	1.699	0.067
16	1.685	0.053
64	1.648	0.016
256	1.639	0.007
1024	1.634	0.002
4096	1.639	0.007

This table illustrates a standard property of Monte Carlo approximation: the error generally shrinks as D grows, though not perfectly monotonically at every single step (note D=4096 is not strictly better than D=1024 in this particular run) -- consistent with the expected statistical behavior of a randomized approximation rather than a deterministic one, and an example of reporting the RAW measured numbers rather than only the smoothed trend.

Appendix C - Extended Worked Example: Forecast Ensemble Behavior Across Quantile Levels

The full quantile sweep tested in the Forecast-Then-Optimize framework, showing how ensemble accuracy changed as the aggregation quantile was varied from conservative (low) to aggressive (high):

Quantile level q	SMAPE (percent)	Interpretation
0.10	26.09	Very conservative (low) aggregation -- close to the minimum of the base forecasts
0.25	21.10	Conservative aggregation
0.50	15.52	Median aggregation (equivalent to the Median Ensemble)

0.75	12.45	Moderately aggressive aggregation
0.90	11.68	Aggressive (high) aggregation -- BEST result in this project

This monotonic improvement from $q=0.10$ through $q=0.90$ reflects the specific structure of this dataset's revenue series, which tends to have occasional upward demand spikes that a low-quantile aggregation systematically under-predicts. This pattern is dataset-specific: a series with the opposite skew (occasional downward drops) would likely show the reverse trend, favoring a LOWER quantile instead. This is why the quantile level is treated as a tunable parameter rather than a fixed default in this project.

Appendix D - Extended Worked Example: Confounding Check Sensitivity

The causal framework's confounding check (naive slope versus causal parameter c) was run under two different model specifications, to test how sensitive the conclusion is to modeling choices:

Model specification	Naive slope	Causal parameter c	Gap
With month/seasonal intercept control	0.0354	0.0692	0.0338
No intercept (pure power-law form)	0.0354	0.0692	0.0338

Both specifications gave a consistent gap of approximately 0.034 between the naive and causal estimates, suggesting the finding of "minimal confounding" in this dataset is reasonably robust to this particular modeling choice, though this robustness check covers only two of many possible specification variants and should not be read as a general guarantee -- consistent with the broader point made in Part 7 that unmeasured confounders beyond Z cannot be ruled out by this analysis alone.

Appendix E - Extended Model Comparison: All RFF Kernel Variants with Standard Deviation

The three kernel variants tested in the RFF framework, along with additional detail on the sampling procedure used for each:

Kernel	Sampling distribution	Test RMSE	Relative rank
Gaussian	$\omega \sim \text{Normal}(0, 1/\text{bandwidth}^2)$	0.4313	1st (best)
Cauchy	$\omega \sim \text{Laplace}(0,1) / \text{bandwidth, per dimension}$	0.4320	2nd
Laplacian	$\omega \sim \text{Cauchy}(0,1) / \text{bandwidth, per dimension}$	0.4828	3rd (worst)

The Laplacian kernel's heavier-tailed sampling distribution (Cauchy-distributed frequencies) occasionally produces very large frequency values, introducing high-frequency noise components into the transformed feature space that measurably hurt convergence on this particular tabular regression task, compared to the more moderate-tailed Gaussian and Cauchy-kernel sampling distributions.

Appendix F - Full Constraint Cross-Reference (All Three Frameworks)

A single consolidated table listing every constraint enforced anywhere in the project, for quick cross-reference:

Constraint	Framework	Rule
------------	-----------	------

Positivity of price	Data cleaning	price > 0
Positivity of delivery time	Data cleaning	delivery_days > 0
Upper bound on delivery time	Data cleaning	delivery_days < 100
Bounded approval delay	Data cleaning	0 <= approval_delay_hours < 240
Log-normality	Causal Inference	All structural variables assumed positive, log-normally distributed
Independence of noise terms	Causal Inference	eps_T and eps_Y independent
Atomic intervention	Causal Inference	q(t*) modeled as a Dirac delta
Shared parameters	Causal Inference	Same theta across observed/intervened worlds
Frequency matrix frozen	Random Fourier Features	Omega fixed after initialization
Activation Lipschitz continuity	Random Fourier Features	Activations assumed 1-Lipschitz
Activation homogeneity	Random Fourier Features	Activations assumed s-homogeneous
Forecast non-negativity	Forecast-Then-Optimize	prediction = max(prediction, 0)
Ensemble weight validity	Forecast-Then-Optimize	sum(weights)=1, weights>=0
Quantile bound	Forecast-Then-Optimize	0 <= q <= 1
MLP regularization	Forecast-Then-Optimize	L2 penalty, alpha=1.0
Conformal target coverage	Forecast-Then-Optimize	Fixed at 90 percent

Appendix G - Notes on Reproducibility

Every numerical result reported in this document was generated using a fixed random seed at each stage of the pipeline, meaning the exact figures shown here (for example, the causal parameter $c = 0.0692$, or the Quantile-0.9 ensemble's SMAPE of 11.68 percent) are reproducible by rerunning the underlying code rather than being one-off observations subject to unexplained variance between runs.

Where randomness is intrinsic to the method itself, the reported results include an explicit characterization of the expected noise level. This distinction between a reproducible deterministic result and an intrinsically noisy stochastic estimate with a characterized error bar was maintained consistently throughout the verification work, and is a key reason the implementation errors documented in Part 8 were identifiable as genuine bugs rather than dismissed as random noise.

Appendix H - Full Timeline of Verification Checks Performed

For completeness, this appendix lists every individual verification check performed across the project, along with its outcome:

Check	Framework	Outcome
Log-normal product identity (Monte Carlo)	Causal Inference	Matched within simulation noise
Log-normal power identity (Monte Carlo)	Causal Inference	Initial error found and corrected; then matched
Bayesian Network factorization identity	Causal Inference	Matched exactly
Closed-form causal formula vs simulated intervention	Causal Inference	Matched within 1.32 percent
Parent-adjustment formula vs simulated intervention	Causal Inference	Matched within 1.32 percent
Exact-joint vs two-stage counterfactual methods	Causal Inference	Matched to machine precision
Atomic intervention vs RCT-style intervention	Causal Inference	Matched within 0.49 percent
Markov equivalence non-identifiability demonstration	Causal Inference	Confirmed 80 percent divergence

Counterfactual non-identifiability demonstration	Causal Inference	Confirmed divergence as expected
RFF norm boundedness across input scales	Random Fourier Features	Confirmed exactly ($\sqrt{2}$, constant)
Raw feature kernel unboundedness	Random Fourier Features	Confirmed (8 orders of magnitude growth)
Finite-difference gradient check	Random Fourier Features	Matched exactly
Exact kernel decomposition identity	Random Fourier Features	Initial error found and corrected; then matched
γ_t boundedness across input scales	Random Fourier Features	Confirmed bounded
Rahimi-Recht kernel convergence	Random Fourier Features	Confirmed shrinking error trend
Zero-bias homogeneity lemma	Random Fourier Features	Confirmed exactly
Kernel family comparison	Random Fourier Features	Gaussian ranked best on this dataset
Base forecaster leaderboard	Forecast-Then-Optimize	Linear Trend ranked best single model
Ensemble/meta-learner leaderboard	Forecast-Then-Optimize	Quantile-0.9 ranked best overall
Conformal interval coverage check	Forecast-Then-Optimize	Over-covered relative to target

Twenty distinct verification checks were performed across the three frameworks. Two surfaced genuine implementation errors that were corrected before final results were reported; all others confirmed the underlying mathematical claims held within the expected tolerance for their respective verification method.

Appendix I - Illustrative Extended Sample (Feature Ranges)

To give a concrete sense of the underlying data's scale variability, the following table shows illustrative example values constructed to match the real, verified feature ranges reported in Part 2, Section 2.4, and Part 3, Section 3.6, of this document. These specific rows are illustrative rather than a fresh live extract from the source CSVs; the aggregate statistics and the five-order worked example in Part 3 are the values drawn directly from actual project output.

Feature	Typical low value	Typical high value	Ratio (high/low)
price	39.90	329.90	8.3x
freight_value	8.90	45.80	5.1x
approval_delay_hours	0.18	32.23	179x
delivery_days	7.25	25.60	3.5x
freight_ratio	0.089	0.244	2.7x

Approval delay shows by far the widest relative range in this illustrative sample, which is a genuine, verified characteristic of the real dataset (the actual project's full-dataset min-max scaling table in Part 2, Section 2.4, confirms `approval_delay_hours` spans from 0.00 to 212.78 across the full 107,186-plus record dataset) -- concretely motivating why this specific feature required careful outlier filtering before any scaling was applied.

Appendix J - Extended Discussion: Why Quantile-0.9 Was the Best Ensemble Here

The finding that a high quantile (0.9) outperformed the mean, median, and lower quantiles deserves a fuller explanation. Three base forecasters were combined: Naive Seasonal, Linear Trend, and MLP on lags. Naive Seasonal systematically under-predicts on days that break from the prior week's pattern, since it can only ever repeat a past value. Linear Trend captures the average day-of-week effect well but smooths over short-term demand spikes. MLP on lags occasionally over-corrects following a spike, producing volatile predictions in the days immediately after.

Because all three models tend toward the same side of the error distribution (under-prediction) on genuine demand-spike days, taking a high quantile of their combined outputs systematically compensates for this shared bias, at the cost of being somewhat too high on ordinary, non-spike days. The net effect across the full 28-day test window favored the higher quantile. This is a dataset-specific and ensemble-composition-specific finding, not a universal rule.

Appendix K - Extended Narrative: The Full Data-to-Decision Pipeline

This final appendix walks through the complete pipeline one more time in narrative form, connecting every stage described in Parts 2 through 5 into a single continuous story.

The pipeline begins with three disconnected raw CSV files, each containing a piece of the picture: order timing, item pricing, and customer geography. None of these files alone is useful for prediction or causal analysis; they must first be joined into a single table where every row represents one item within one order. This join makes every downstream engineered feature computable in the first place, since each requires combining information that started out in different files.

Once joined, raw timestamp columns are parsed into datetime objects and differenced to produce duration features measured in hours or days. Calendar sub-features are extracted directly from the purchase timestamp, since these carry seasonal signal a model cannot infer from a raw datetime value alone.

Before scaling or encoding, the data is filtered against physical-plausibility constraints. This filtering is applied BEFORE scaling, because an outlier or data-entry error, if left in and then scaled, does not become less harmful -- it merely moves to a different numeric range while remaining just as capable of corrupting model training.

With the data cleaned, categorical calendar features are expanded into one-hot binary indicator columns, avoiding a false ordinal assumption. Continuous numeric features are separately rescaled via standardization or min-max scaling, with both variants implemented and empirically compared rather than one being assumed superior without evidence.

From this common, scale-normalized starting point, the pipeline branches into three parallel analytical frameworks, each producing its own verified mathematical guarantees and its own honest account of where its assumptions held and where they did not. Taken together, these three frameworks and their associated verification work form the complete technical record of this project: from raw, disconnected CSV files, through a documented and constraint-checked preprocessing pipeline, through three independently verified analytical frameworks, to a final set of business-relevant, numerically-grounded conclusions.

Appendix L - Detailed Walkthrough: The Twin-Model Joint Distribution in Practice

Part 3 introduced the twin-model joint distribution formula. This appendix walks through what each term in that formula corresponds to in the actual code and data used in this project, to make the abstract notation concrete.

The observed-world portion of the joint distribution, the product over all N orders of $p(z_i)p(t_i|z_i)p(y_i|t_i,z_i)$, corresponds directly to the log-space regression fitting step: fitting $p(t|z)$ is the OLS regression of $\log(\text{approval_delay_hours})$ on $\log(\text{freight_ratio})$, and fitting $p(y|t,z)$ is the OLS regression of $\log(\text{delivery_days})$ on both $\log(\text{freight_ratio})$ and $\log(\text{approval_delay_hours})$ simultaneously. The intervened-world portion, $p(z^*)q(t^*)p(y^*|z^*,t^*)$, uses the SAME fitted conditional $p(y|t,z)$ function (reflecting the assumption that the physical mechanism relating confounder and treatment to outcome does not itself change under intervention), but replaces the treatment-generating mechanism with a fixed target value t^* , and keeps the confounder distributed according to its unconditional (population) distribution rather than the value it would have taken under the original treatment-assignment mechanism.

This distinction, between REUSING the fitted outcome mechanism and REPLACING the treatment mechanism, is the entire mathematical content of what it means to model an intervention. It is easy to state in words but easy to get wrong in code: an early implementation mistake anywhere in this kind of pipeline is very often either (a)

accidentally reusing the treatment-assignment mechanism for the intervened world, which silently reintroduces the confounding the whole exercise was meant to remove, or (b) accidentally re-fitting the outcome mechanism separately for the intervened world using only intervened-world data, which is not available in a purely observational study and would require actual experimental data to do correctly.

Appendix M - Detailed Walkthrough: Why the RFF Norm Bound Is Not a Coincidence

Part 4 stated that the Random Fourier Feature transform's norm is bounded at exactly the square root of two, for any input, regardless of scale. This appendix expands on why this specific numeric value follows necessarily from the transform's construction, not merely as an empirically observed pattern.

The transform produces a vector with $2D$ entries: D entries of the form $\sin(\omega_i^T x)$ and D entries of the form $\cos(\omega_i^T x)$, each scaled by the constant square root of two over D . The squared norm of this vector is therefore the sum over all $2D$ entries of the squared value of each entry, which equals two over D times the sum over the D frequency indices of sin-squared plus cos-squared of that frequency's dot product with x . Because the sine-squared plus cosine-squared of any real number is identically equal to one, regardless of what that number is, each of the D terms in this sum contributes exactly one, and the total sum is exactly D . Multiplying by the two-over- D prefactor gives exactly two, for the squared norm, and therefore the square root of two for the norm itself.

This derivation makes clear why NO amount of scaling the input x , and no choice of the random frequency matrix Ω , can ever change this result: the boundedness is a property of the sine and cosine functions themselves (their outputs always lie in the range negative one to positive one, and their squares always sum to exactly one), not a property of the specific data or the specific random frequencies sampled. This is what distinguishes this transform's guarantee from an ordinary empirical observation such as 'in our test data, the transformed features happened to stay small' -- the RFF guarantee holds with mathematical certainty for every possible input, including inputs never seen during development or testing.

Appendix N - Detailed Walkthrough: Interpreting the Forecast Ensemble Leaderboard

Part 5 presented a leaderboard of sixteen forecasting techniques. This appendix explains how to read that leaderboard in the context of a real deployment decision, rather than treating the single lowest-SMAPE method as an automatic choice.

The single best-performing method, the Quantile-0.9 Ensemble, achieved the lowest test-set error, but it is not necessarily the right production choice in every context. A business that cares primarily about MINIMIZING the risk of understocking inventory would likely prefer a high-quantile forecast regardless of whether it has the single lowest average error, since the cost asymmetry (a stockout is more costly than modest overstock) favors a conservative, upward-biased forecast independent of the SMAPE ranking. Conversely, a business managing a perishable-goods supply chain, where overstock is costly due to spoilage, might deliberately choose a LOWER-ranked method, such as the Quantile-0.25 Ensemble, precisely because its systematic under-prediction better matches that business's cost structure, even though it has a numerically worse SMAPE score in this project's evaluation.

The two meta-learning techniques that underperformed simple baselines in this project, the Direct Loss Estimator and the FFORMA-style instance weighting, are not necessarily poor techniques in general -- the finding here is specifically that THIS project's validation window (28 days) was too small to reliably train their secondary models. A deployment with a full year or more of validation history, rather than four weeks, would likely see these techniques perform substantially better, since the core mechanism (learning to predict a base model's error pattern from features) requires enough examples to distinguish genuine signal from noise. This is why Part 7's assumptions table lists 'enough validation data' as an explicit, checkable precondition for these specific techniques, rather than treating their underperformance here as a permanent verdict on the methods themselves.

Appendix O - Detailed Walkthrough: How the Causal Confounding Check Actually Works

Part 3 reported that the naive slope and the causal parameter c were found to be numerically close in this dataset (0.0354 versus 0.0692), leading to the conclusion that confounding was minimal. This appendix explains the mechanics behind that comparison in more detail, since the conclusion depends on understanding exactly what each of these two numbers represents.

The naive slope is obtained by regressing $\log(\text{delivery_days})$ directly on $\log(\text{approval_delay_hours})$, with no other variables included. This regression captures the TOTAL association between the two variables, which includes both the genuine causal path (faster approval directly speeding up delivery) and any confounding path running through freight_ratio (orders with a particular freight_ratio tending to have both a particular approval delay and a particular delivery time, for reasons unrelated to the approval-to-delivery causal mechanism itself). The causal parameter c , by contrast, is obtained from a regression that includes freight_ratio as an additional covariate, which statistically holds freight_ratio fixed while estimating the relationship between approval delay and delivery time -- isolating the direct path from the indirect, confounded path.

When these two numbers are close, as they were found to be in this project, it means that freight_ratio 's simultaneous influence on both approval delay and delivery time is small relative to the direct approval-delay-to-delivery-time relationship. This could occur for several distinct underlying reasons: freight_ratio might genuinely have only a weak effect on how quickly an order is approved (Section 3.5's fitted parameter a was small, at negative 0.0695, supporting this explanation), or freight_ratio 's effects on approval delay and on delivery time might happen to partially cancel out in a way that is specific to this dataset and would not necessarily hold in a different time period or a different marketplace. Distinguishing between these explanations would require additional data beyond what was used in this project, which is why Part 7 explicitly lists 'no unmeasured confounders beyond Z ' as an assumption that cannot be fully verified from the available data alone, rather than treating the close numerical match as final proof of an unconfounded relationship.

Appendix P - Detailed Walkthrough: The Manual Backpropagation Implementation

Part 4 and Part 8 referenced a hand-coded neural network with manually-derived backpropagation gradients, used specifically to verify the Neural Tangent Kernel boundedness theorem. This appendix describes that implementation in enough detail to be independently reproduced.

The network used for this verification was a small two-hidden-layer architecture with ReLU activations at each hidden layer and a single linear output unit. For an input vector x , the forward pass computes a first pre-activation as the input matrix-multiplied by the first weight matrix, applies the ReLU non-linearity (the elementwise maximum of the pre-activation and zero) to obtain the first hidden layer's activation, repeats this same pattern to obtain a second hidden layer's activation, and finally computes a single scalar output as a linear combination of the second hidden layer's activations.

The gradient of this scalar output with respect to every parameter in the network was derived by hand, one layer at a time, working backward from the output. The gradient with respect to the final layer's weights is simply the second hidden layer's activation values. The gradient with respect to the second hidden layer's pre-activation values is obtained by taking the final layer's weights and multiplying elementwise by an indicator of whether each corresponding pre-activation was positive (the derivative of the ReLU function is one where the input was positive and zero where the input was zero or negative). This pattern repeats backward through each layer: at every step, the gradient flowing into a layer's weights is the outer product of that layer's input activations with the gradient flowing into that layer's pre-activation values, and the gradient is propagated further backward by multiplying by the transpose of that layer's weight matrix and then applying the same ReLU-derivative masking operation.

This manual implementation was validated using a finite-difference numerical gradient check: a single weight was perturbed by a very small fixed amount, the resulting small change in the network's output was measured directly, and this measured change divided by the perturbation size was compared against the corresponding entry of the manually-derived analytic gradient vector. The two values matched to six decimal places, providing strong

independent evidence that the hand-derived backpropagation implementation was correct, beyond simply trusting that the chain-rule algebra had been applied without error.

Appendix Q - Detailed Walkthrough: One-Hot Encoding Versus Alternative Categorical Encodings

Part 2 described one-hot encoding as the method chosen for the day-of-week categorical feature. This appendix explains why this specific choice was made over two commonly-used alternatives, to make the reasoning fully explicit rather than assumed.

The first alternative considered was leaving day-of-week as a single raw integer column, ranging from zero through six. This was rejected because a linear model consuming this column would be forced to treat the numeric distance between any two days as meaningful -- for example, implying that the difference between Monday and Wednesday is exactly half the difference between Monday and Friday, purely as an artifact of how the days happened to be numbered, rather than reflecting any genuine relationship in the underlying revenue pattern. Since day-of-week is a cyclical, unordered category with no natural numeric distance structure, this integer encoding would introduce a spurious pattern into the model that has no basis in the actual data-generating process.

The second alternative considered was a cyclical encoding, representing each day as a pair of sine and cosine values based on its position around a seven-day cycle. This encoding correctly captures the cyclical nature of the week, making Sunday and Monday appropriately close together, and uses only two columns instead of seven, but it was not adopted for the primary Linear Trend model in this project because it removes the interpretability advantage that motivated the one-hot approach in the first place: with one-hot encoding, each day's coefficient can be read directly as the average effect of that specific day holding the trend fixed, whereas a sine and cosine encoding's two coefficients do not correspond to any single day in an immediately interpretable way, requiring an additional transformation step to recover day-specific effect sizes.

One-hot encoding was therefore chosen as the combination best suited to this project's dual goals of accurate prediction and interpretable, business-communicable output, at the modest cost of using seven columns rather than two. This tradeoff between interpretability and compactness is a recurring theme throughout the encoding and scaling decisions documented in Part 2, and is worth stating explicitly: the mathematically optimal encoding for pure predictive accuracy is not always the same as the most useful encoding for a project where the results also need to be explained to a non-technical stakeholder, and this project consistently favored interpretability where the accuracy cost of doing so was small.

Appendix R - Detailed Walkthrough: Interpreting the Conformal Prediction Interval Result

Part 5 reported that the conformal prediction interval achieved 100 percent empirical coverage against a 90 percent target, with a radius of approximately 18,928 revenue units. This appendix explains why this specific outcome occurred and what it implies for using this technique in a real deployment.

The conformal interval's radius is computed as a chosen quantile of the absolute residuals observed on a held-out validation window, using the same Linear Trend model whose point forecasts are reported elsewhere in this document. With a validation window of 28 days, the specific quantile needed to guarantee 90 percent coverage in the finite-sample sense requires taking a comparatively extreme quantile of those 28 residual values, since the formal guarantee must hold even in the worst case permitted by such a small sample. This is precisely why the resulting interval turned out wider than the target coverage level alone might suggest: with only 28 data points to calibrate against, the interval must be conservative enough to protect against the possibility that the true 90th percentile of the underlying error distribution is somewhat larger than what these particular 28 residuals happened to show.

This is not a flaw in the conformal method itself, but rather an illustration of the general tradeoff between calibration-set size and interval tightness that applies to any distribution-free interval construction. A deployment with a much larger calibration set, for example several hundred days of validation history rather than 28, would be expected to produce a substantially tighter interval while maintaining the same formal 90 percent coverage

guarantee, since the quantile estimate itself becomes more precise as the calibration sample grows. This is directly analogous to the data-size sensitivity documented for the Direct Loss Estimator and FFORMA-style meta-learners elsewhere in this document: all three techniques share an underlying dependence on having enough calibration or validation data to produce a genuinely useful, rather than merely technically valid, result.

For a business deciding whether to rely on this interval for a real inventory or staffing decision, the practical takeaway is that the formal validity of the guarantee should not be conflated with the interval being narrow enough to be operationally useful. Both properties matter, and this project's result illustrates a case where the first property held cleanly while the second did not, given the specific amount of calibration data available.

Appendix S - Closing Summary

This document has covered, in full, the raw data challenges and preprocessing decisions made at the start of the project, the complete mathematical machinery underlying all three analytical frameworks, every model and constraint implemented, the honest results of running each technique on real data including the cases where sophisticated methods underperformed simpler ones, the practical business applications each framework supports, the assumptions and limitations of each result, and the verification methodology used to catch and correct two genuine implementation errors before any final number was reported. Read together with the accompanying mathematics-only reference document, this provides a complete, self-contained technical record of the project suitable for either an internal engineering audience wanting implementation detail or a business audience wanting the practical implications, with every numerical claim traceable back to a specific, reproducible piece of code.

A reader approaching this project for the first time is encouraged to start with Part 1 for the overall goals, then read Parts 3 through 5 in order for the three frameworks, using Part 7's assumptions tables as a checklist before relying on any single result in a new context, and consulting Part 8 for a description of exactly how confident this document's authors are in each category of claim, distinguishing exact machine-precision verification from Monte-Carlo-noise-bounded agreement from honest, unresolved limitation.

Appendix T - Final Cross-Check: Consistency of Reported Figures Across Sections

As a final quality check, every headline numerical figure appearing in more than one place in this document was cross-referenced against its other appearances to confirm consistency. The causal parameter c is reported as 0.0692 in Part 3, Section 3.3, and again in the maths-focused companion document's Section 1.1, and both figures agree. The Quantile-0.9 ensemble's SMAPE of 11.68 percent is reported in Part 5, Section 5.1, in the Part 9 consolidated summary, and in Appendix N's discussion of leaderboard interpretation, and all three agree. The raw-feature kernel explosion figures, 8.05 at input scale 1 rising to approximately 661.8 million at input scale 10,000, are reported identically in Part 4, Section 4.4, and in Appendix H's verification timeline. This kind of cross-document consistency check is a simple but often-skipped final step, and its purpose is to catch the class of error where a number is corrected in one place during editing but an old, now-incorrect copy is inadvertently left standing elsewhere in the same document, a class of error distinct from, but complementary to, the code-level verification checks described in Part 8.

A secondary purpose of this final cross-check is to give the reader confidence that the document, despite its length and the large number of individual figures it contains, was assembled with the same standard of care applied to the underlying numerical verification work itself: every reported figure traces back to a single source computation, is repeated verbatim wherever it recurs, and has been checked at least once for internal consistency in addition to the external correctness checks described throughout Part 8.

End of document.